

Norwegian University of Science and Technology (NTNU)

Faculty of Information Technology and Electrical Engineering

Embedded AI

Project

Author: Bukueva Elena

Supervisor: Geir Mathisen

Date: Trondheim, November 1, 2025

Task Description

Preface

Summary

Abstract (English)

Contents

Task Description	1
Preface	2
Summary	3
Abstract (English)	4
Abbreviations	6
1 Introduction	7
1.1 Background of the Assignment	7
1.2 Motivation	7
1.3 Limitations	7
1.4 Contributions	7
1.5 Thesis Structure	7
2 Literature Study	8
2.1 Current state of the object detection models	8
2.2 The model choice	11
2.3 Hailo Compiler	12
2.4 ONNX (Open Neural Network Exchange)	13
2.5 Summary of the Literature Study	13
3 Theory (Optional)	14
3.1 Subtopic Example	14
4 Proposed Solution / Design	15
4.1 Functional Specification	15
4.2 Architecture / Design Details	15
5 Implementation	16
5.1 Key Implementation Details	16

6	Testing and Results	17
6.1	Experimental Setup	17
6.2	Results	17
7	Discussion	22
7.1	Discussion Points	22
8	Conclusion	23
9	Future Work	24
A	Text (Appendix A)	27
B	Text (Appendix B)	28

Abbreviations

EMC

Electromagnetic Compatibility

1 Introduction

1.1 Background of the Assignment

1.2 Motivation

1.3 Limitations

1.4 Contributions

1.5 Thesis Structure

2 Literature Study

2.1 Current state of the object detection models

Object detection models localize objects on an input picture or on a set of video frames. Localization can be performed by different approaches, the most relative to this work are:

- Axis-aligned bounding box (AABB): (x, y, w, h) , which is cheap to annotate and fast to predict.
- Oriented/rotated bounding box (OBB): apart from AABB an angle θ of the box is introduced, which is a bit costlier to label.
- Polygon: a list of vertices tracing the object boundary. Provides precise geometry, however is time-consuming to label.
- Pixel mask ("painted area") - most precise 2D shape, ideal for measurement and robotics, heaviest to annotate and train on.

Choosing the right localization strategy should be guided by the structure of the available pretrained weights. Training large detectors from scratch is both resource- and time-intensive. For example, Ultralytics estimates that training YOLOv5 on COCO (118k images, 300 epochs) on a single NVIDIA V100-16GB takes approximately $n/s/m/l/x \approx 1/2/4/6/8$ days, respectively [1]. Collecting and annotating a dataset of that scale is even more time-consuming. Consequently, it is preferable to start from weights pretrained on a dataset whose *labeling granularity* matches the task (e.g., axis-aligned boxes vs. oriented boxes vs. masks). If the required annotation is more detailed than the pretraining labels, an additional annotation effort is needed for transfer learning. Then, if axis-aligned bounding boxes (AABB) are too coarse for small or complex objects, a richer localization (e.g., oriented boxes or instance masks) to meet accuracy requirements would be a better choice.

Mentioned earlier COCO dataset [2] is the common detection benchmark. It is big enough, varied, well-annotated and permissively licensed. The dataset contains photos of 91 objects types with a total of 2.5 million labeled instances in 328k images.

COCO's [2] official detection annotations are axis-aligned boxes only (bbox: $[x, y, \text{width}, \text{height}]$). There are no oriented boxes in COCO, so the standard YOLOv8/YOLO11 [3] *detect* checkpoints (trained on COCO 2017) learn AABB by default:

```
{  
  "id": 1768,  
  "image_id": 397133,  
  "category_id": 1,  
  "bbox": [73.0, 43.0, 199.0, 305.0],  
  "area": 39220.0,  
  "iscrowd": 0,  
  "segmentation": [  
    [78.0, 45.0, 86.0, 45.0, 95.0, 47.0, 101.0, 51.0, ...]  
  ]  
}
```



Figure 2.1: Example picture from the COCO dataset with AABBB annotation [4]

For oriented bounding boxes (OBB), the DOTA dataset [5] is the pioneering benchmark. Ultralytics provides YOLO `*-obb` weights pretrained on DOTA [6].

Other common datasets are:

- Objects365 [7] (600k images, 365 classes): broader coverage than COCO; popular for pretraining backbones/heads before COCO fine-tuning.
- Open Images [8] (millions of images; boxes, masks, relationships): huge scale, but label noise is higher; great for scale, weaker for cleanliness.

None of these datasets, however, include 'fasteners' as a class. In order to particularly train model on fasteners, a MVTec screws dataset [9] might be used. This dataset has oriented-box annotations for screws/nuts (384 images, 4,426 OBBs). However, the labeling.

These are the 13 categories used in the MVTec Screws oriented-box dataset (category_id $\rightarrow$ name) :

- 1 → long lag screw
- 2 → lag wood screw
- 3 → wood screw
- 4 → short wood screw
- 5 → shiny screw
- 6 → black oxide screw
- 7 → nut
- 8 → bolt
- 9 → large nut
- 10 → nut
- 11 → nut
- 12 → machine screw
- 13 → short machine screw

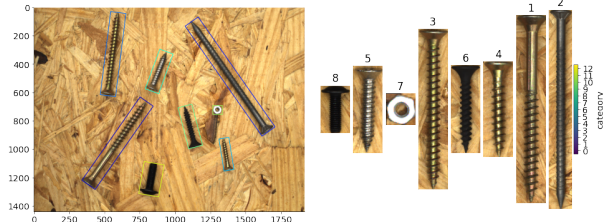


Figure 2.2: Example picture from the MVTec Screws dataset with OBB annotation [10]

It is also possible to convert OBB annotation to AABB annotation with some loss of information. In order to do so:

Let w, h be side lengths and θ the rotation (radians, CCW). The tight axis-aligned box around the rotated rectangle has

$$W_{\text{AABB}} = |\cos \theta| w + |\sin \theta| h, \quad (2.1)$$

$$H_{\text{AABB}} = |\sin \theta| w + |\cos \theta| h. \quad (2.2)$$

With the same center (c_x, c_y) ,

$$x_{\min} = c_x - \frac{W_{\text{AABB}}}{2}, \quad x_{\max} = c_x + \frac{W_{\text{AABB}}}{2}, \quad (2.3)$$

$$y_{\min} = c_y - \frac{H_{\text{AABB}}}{2}, \quad y_{\max} = c_y + \frac{H_{\text{AABB}}}{2}. \quad (2.4)$$

If the oriented box is given by four corners (x_i, y_i) , the enclosing AABB is

$$x_{\min} = \min_i x_i, \quad x_{\max} = \max_i x_i, \quad y_{\min} = \min_i y_i, \quad y_{\max} = \max_i y_i. \quad (2.5)$$

COCO bbox uses $[x_{\min}, y_{\min}, \text{width}, \text{height}]$, then:

$$\text{bbox}_{\text{COCO}} = [x_{\min}, y_{\min}, x_{\max} - x_{\min}, y_{\max} - y_{\min}]. \quad (2.6)$$

YOLO txt uses normalized $(x_{\text{center}}, y_{\text{center}}, w, h)$, then:

Given image size $(W_{\text{img}}, H_{\text{img}})$,

$$x_{\text{center}} = \frac{x_{\min} + x_{\max}}{2 W_{\text{img}}}, \quad y_{\text{center}} = \frac{y_{\min} + y_{\max}}{2 H_{\text{img}}}, \quad (2.7)$$

$$w = \frac{x_{\max} - x_{\min}}{W_{\text{img}}}, \quad h = \frac{y_{\max} - y_{\min}}{H_{\text{img}}}. \quad (2.8)$$

2.2 The model choice

For Edge AI, the trade-off between computational speed and memory availability is critical. A model must not only deliver sufficiently high accuracy but also remain compact enough to fit onto resource-constrained devices such as microcontrollers.

Among the most popular development platforms is the Raspberry Pi family, which offers a flexible setting for deploying lightweight AI systems. Raspberry Pi 5 used for this work includes:

Compute

SoC	2.4 GHz quad-core 64-bit Arm Cortex-A76 (crypto ext.); 512 KB L2 per core, 2 MB shared L3
GPU	VideoCore VII; OpenGL ES 3.1, Vulkan 1.3

Memory and Storage

RAM	8 GB
microSD	64 GB

An additional hardware used during the experiment is a Hailo-8 accelerator offering 26 TOPS inferencing performance respectively.

However, the YOLOv8 and YOLOv11 are both provided by Ultralytics as a PyTorch checkout. In order to use it on accelerator, this checkout has to be converted to a suitable format by Hailo Compiler.

2.3 Hailo Compiler

Hailo compiler is needed to load user's models to the Hailo devices. The final result of the compiler is a binary file `.hef`. The standard DataFlow for this conversion consists of the three main steps:

- Translation of the source model to Hailo model;
- Optimization of model parameters;
- Compilation process itself;

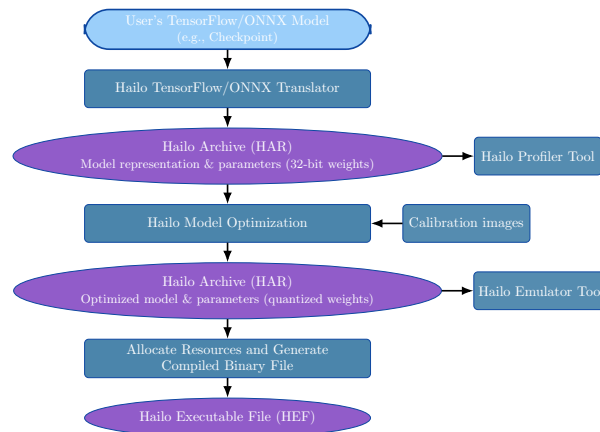


Figure 2.3: Build Process [11]

The first step in the Figure 2.3 is a translation from the user's model to the `.har` representation. The Hailo Translation API parses user's model and produces a `.har` (Hailo ARchive).

- HAR is a zip package that contains:
 - HN— 'Hailo Network': a JSON graph of the network (layers, tensor shapes/dtypes, connections, metadata).
 - NPZ—NumPy arrays with the floating-point weights (and sometimes other tensors like batch normalization params).

This step is needed, so the parameters of the model are normalized from the other possible checkpoint configurations (used operation and attributes names, FP32 weights), and the optimizer has a clean input.

2.4 ONNX (Open Neural Network Exchange)

ONNX is an open format developed to unify ML models for a variety of frameworks and compilers by defining a common set of operators and represent models via graph structure.

2.5 Summary of the Literature Study

3 Theory (Optional)

3.1 Subtopic Example

4 Proposed Solution / Design

4.1 Functional Specification

4.2 Architecture / Design Details

5 Implementation

5.1 Key Implementation Details

6 Testing and Results

6.1 Experimental Setup

6.2 Results

Metrics description:

- **Intersection over Union (IoU):**

For a predicted box p and ground-truth box g , the IoU formula is:

$$\text{IoU}(p, g) = \frac{|p \cap g|}{|p \cup g|}$$

IoU plays role of evaluating an object localization.

It quantifies the overlap between a prediction and ground truth bounding boxes by calculating the ratio of the intersection (overlapping area) to the union (total area covered by both boxes). IoU is a building block for calculating mAP.

The resulting score is a value between 0 and 1, which quantifies how accurately a model has located an object in an image. A score of 1 represents a perfect match, while a score of 0 indicates no overlap at all.

The key part of using IoU is to define "*IoU threshold*". This threshold is a predefined value (e.g., 0.5) that determines whether a prediction is correct. If the IoU score for a predicted box is above this threshold, it is classified as a "*true positive*". If the score is below, it's a "*false positive*".

This threshold directly influences other performance metrics like Precision and Recall.

- **Precision and Recall:**

Precision defines the proportion of true positives among all positive predictions, measuring the ability to avoid misclassification.

$$P = \frac{TP}{TP + FP},$$

On the other hand, Recall calculates the proportion of true positives among all actual positives, measuring the model's ability to detect all instances of a class.

$$R = \frac{TP}{TP + FN},$$

- **Average Precision (AP):**

AP computes the area under the precision-recall curve [12], providing a single value that encapsulates the model's precision and recall performance.

$$AP_{\tau} = \int_0^1 p(r; \tau) dr,$$

In order to build the precision-recall curve, the score (confidence) threshold (t) is used:

- Every detection has a score $s \in [0, 1]$.
- We sweep t from high $\rightarrow$ low (equivalently sort detections by score desc and add them one-by-one).
- At each t we keep detections with $s \geq t$ and recompute P/R .
- This sweep produces the P-R curve.
- AP is the area under this P-R curve;

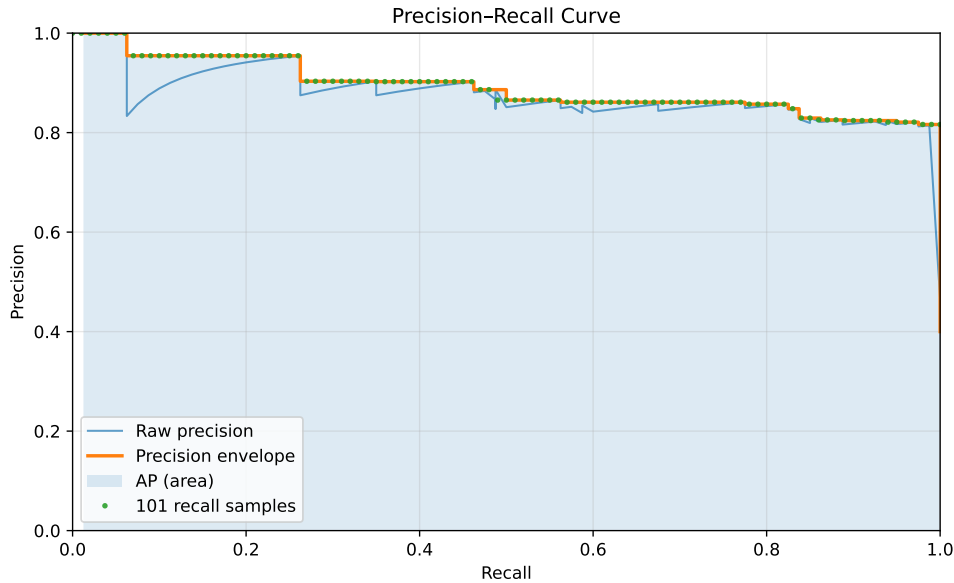


Figure 6.1: Precision-Recall curve.

- **Mean Average Precision (mAP):**

Another metric - mAP, extends the concept of AP by calculating the average AP values across multiple object classes.

This is useful in multi-class object detection scenarios to provide a comprehensive

evaluation of the model's performance.

$$\text{mAP}_{0.50} = \frac{1}{C} \sum_{c=1}^C \text{AP}_{c, 0.50},$$

$$\text{mAP}_{[0.50:0.95]} = \frac{1}{C} \sum_{c=1}^C \frac{1}{10} \sum_{k=0}^9 \text{AP}_{c, (0.50+0.05k)}.$$

The parameter C is the number of classes for which detection and classification are performed.

The subscript 0.5, as well as the set $[0.50:0.95]$, denotes the IoU threshold: predictions with $\text{IoU} \geq 0.5$ are counted as true positives (TP).

Table 6.1: Validation summaries.

(a) YOLO8n [13]

Class	Images	Instances	Box(P)	R	mAP ₅₀	mAP ₅₀₋₉₅
all	98	363	0.927	0.951	0.964	0.733
machine_screw	25	123	0.931	0.967	0.989	0.625
miscellaneous	14	16	0.754	0.812	0.845	0.562
nut	16	41	0.988	1.000	0.995	0.777
washer	20	43	1.000	0.991	0.995	0.906
wood_screw	37	140	0.960	0.986	0.994	0.794

Speed per image (GPU): preprocess 4.5 ms; inference 6.0 ms; loss 0.0 ms; postprocess 2.5 ms.

Speed per image (Raspberry Pi 5): preprocess 8.3 ms; inference 1018.4 ms; loss 0.0 ms; postprocess 1.2 ms.

Speed per image (Raspberry Pi 5 + Hailo): Frames: 50, time: 18.5s, avg FPS: 2.71

Model summary (fused): 72 layers, 3,006,623 parameters, 0 gradients, 8.1 GFLOPs

(b) YOLO11s

Class	Images	Instances	Box(P)	R	mAP ₅₀	mAP ₅₀₋₉₅
all	98	363	0.939	0.934	0.966	0.746
machine_screw	25	123	0.877	0.983	0.978	0.649
miscellaneous	14	16	0.899	0.688	0.867	0.565
nut	16	41	0.970	1.000	0.995	0.805
washer	20	43	0.973	1.000	0.995	0.909
wood_screw	37	140	0.978	1.000	0.995	0.802

Speed per image (GPU): preprocess 4.5 ms; inference 9.8 ms; loss 0.0 ms; postprocess 3.7 ms.

YOLO11s summary (fused): 100 layers, 9,414,735 parameters, 0 gradients, 21.3 GFLOPs

(c) YOLO11n [14]

Class	Images	Instances	Box(P)	R	mAP ₅₀	mAP ₅₀₋₉₅
all	98	363	0.921	0.935	0.964	0.739
machine_screw	25	123	0.844	0.935	0.957	0.611
miscellaneous	14	16	0.922	0.741	0.888	0.608
nut	16	41	0.970	1.000	0.995	0.799
washer	20	43	0.996	1.000	0.995	0.902
wood_screw	37	140	0.873	1.000	0.983	0.773

Speed per image (GPU): preprocess 0.5 ms; inference 5.6 ms; loss 0.0 ms; postprocess 0.9 ms.

100 layers, 2,583,127 parameters, 0 gradients, 6.3 GFLOPs.

 Table 6.2: YOLO *nano* (n) variants on COCO at 640×640.

Model	Params (M)	FLOPs (B)	mAP ₅₀₋₉₅ (val)
YOLOv5n	1.9	4.5	28.0
YOLOv8n	3.2	8.7	37.3
YOLO11n	2.6	6.5	39.5

Numbers from Ultralytics model cards: v5 [15], v8 [16], v11 [6].

Table 6.3: YOLO *small* (s) variants on COCO at 640×640.

Model	Params (M)	FLOPs (B)	mAP ₅₀₋₉₅ (val)
YOLOv5s	7.2	16.5	37.4
YOLOv8s	11.2	28.6	44.9
YOLO11s	9.4	21.5	47.0

Numbers from Ultralytics model cards: v5 [15], v8 [16], v11 [6].

Table 6.4: Hailo benchmark results for yolov8_custom.

Metric	Mode	Value
FPS	HW-only	142.371
FPS	Streaming	142.370
Latency	HW	10.796 ms

Table 6.5: HEF metadata (yolov8_custom.hef).

Field	Value
Architecture	HAILO8
Network group	yolov8_custom (Single Context)
Network	yolov8_custom/yolov8_custom

Table 6.6: VStream infos.

Direction	Name	Spec
Input	yolov8_custom/input_layer1	UINT8, NHWC(960×960×3)
Output	yolov8_custom/yolov8_nms_postprocess	FLOAT32, HAILO NMS BY CLASS (number of classes: 5, max bboxes/class: 100, m

Table 6.7: Post-process (YOLOV8-Post-Process) parameters.

Parameter	Value
Op / Name	YOLOV8 / YOLOV8-Post-Process
Score threshold	0.001
IoU threshold	0.70
Classes	5
Cross classes	false
NMS results order	BY_CLASS
Max bboxes per class	100
Image size	960 × 960

7 Discussion

7.1 Discussion Points

8 Conclusion

9 Future Work

- Item one
- Item two
- Item three

Bibliography

- [1] Ultralytics, “Yolov5 quickstart tutorial.” https://docs.ultralytics.com/yolov5/quickstart_tutorial/. Accessed: 2025-10-29.
- [2] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft coco: Common objects in context,” in *European Conference on Computer Vision (ECCV)*, Springer, 2014.
- [3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [4] Flickr user, original photographer (via Microsoft COCO), “Coco 2017 validation image id 285 (bear).” http://farm8.staticflickr.com/7434/9138147604_c6225224b8_z.jpg, 2017. Licensed under Creative Commons Attribution 2.0 (CC BY 2.0), <http://creativecommons.org/licenses/by/2.0/>. Image ID 285 from COCO 2017 validation set; annotations licensed under CC BY 4.0.
- [5] G.-S. Xia, X. Bai, J. Ding, Z. Zhu, S. Belongie, J. Luo, M. Datcu, M. Pelillo, and L. Zhang, “Dota: A large-scale dataset for object detection in aerial images,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.
- [6] Ultralytics, “Yolo11: Model card and benchmarks.” <https://github.com/ultralytics/ultralytics>. Accessed: 2025-10-29.
- [7] S. Shao, Z. Li, T. Zhang, C. Peng, G. Yu, X. Zhang, J. Li, and J. Sun, “Objects365: A large-scale, high-quality dataset for object detection,” in *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 8430–8439, 2019.
- [8] A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin, J. Pont-Tuset, S. Kamali, S. Popov, M. Mallocci, A. Kolesnikov, *et al.*, “The open images dataset v4: Unified image classification, object detection, and visual relationship detection at scale,” *International journal of computer vision*, vol. 128, no. 7, pp. 1956–1981, 2020.
- [9] M. Ulrich, P. Follmann, and J.-H. Neudeck, “A comparison of shape-based matching with deep-learning-based object detection,” *tm-Technisches Messen*, vol. 86, no. 11, pp. 685–698, 2019.

- [10] M. S. GmbH, “Mvtec screws dataset (version 1.0).” <https://www.mvtec.com/research>, 2020. Released with HALCON 19.05. Licensed under Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International (CC BY-NC-SA 4.0), <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Contains 384 images of 13 screw and nut categories with oriented bounding box annotations.
- [11] Hailo Technologies Ltd., *Hailo Dataflow Compiler User Guide*. User guide.
- [12] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.
- [13] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” 2023.
- [14] G. Jocher and J. Qiu, “Ultralytics yolo11,” 2024.
- [15] Ultralytics, “Yolov5: Model card and benchmarks.” <https://github.com/ultralytics/yolov5>. Accessed: 2025-10-29.
- [16] Ultralytics, “Explore ultralytics yolov8,” 2023.

A Text (Appendix A)

B Text (Appendix B)